Context-Aware False Positive Reduction in Cloud Microservice Anomaly Detection

Project Report Submitted in Partial Fulfilment of the Requirements for the Degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Pratyush Jain: (Roll No. 2210110970)
Vajra Akurathi: (Roll No. 2210110779)

Under the Supervision of

Dr. Suchi Kumari

SHIV NADAR
INSTITUTION OF EMINENCE DEEMED TO BE
UNIVERSITY
DELHI NCR

Department of Computer Science and Engineering

April 2026

Acknowledgments

We express our sincere gratitude to our supervisor, Dr. Suchi Kumari, whose expert guidance, constructive criticism, and continuous encouragement have been instrumental in shaping the direction and quality of this research. Her expertise in machine learning, distributed systems, and software engineering provided invaluable insights throughout the iterative development of this framework.

We are deeply grateful to Shiv Nadar Institution of Eminence, particularly the School of Engineering, for providing the essential infrastructure, computing facilities, and a vibrant academic environment that made this research possible. The university's commitment to fostering innovation enabled us to pursue an unconventional research methodology—eleven iterative rounds of experimentation across four months—that would not have been possible without institutional support.

Special thanks to the RCAEval benchmark team for providing a standardized evaluation framework with 735 real-world fault cases across three microservice systems. We also acknowledge the open-source communities behind PyTorch, scikit-learn, XGBoost, LightGBM, CatBoost, SHAP, and ruptures that formed the technical foundation of our implementation.

Finally, we acknowledge the broader AIOps research community, whose papers on shortcut learning, FiLM conditioning, and tabular deep learning informed our evaluation methodology and architectural decisions.

Abstract

Modern cloud-native systems built on microservice architectures generate high-volume monitoring alerts with reported false-positive rates of 6–28%. Operational context—scheduled load events, deployment history, and traffic-pattern signals—is routinely collected by production observability stacks but is not used as a feature by existing anomaly detection or attribution systems. This work investigates whether contextual features materially improve anomaly attribution and, if so, whether the benefit is specific to a neural architecture or is a general, reproducible signal.

We first evaluate on the RCAEval benchmark (three microservice systems, three dataset variants, ∼1,500 pooled cases). Gradient-boosted tree baselines dominate all neural variants on real data—consistent with published findings on small tabular tasks. To test whether sample size is the bottleneck, we design a difficulty-aware synthetic benchmark with explicit anti-leakage safeguards (pre-injection splits, randomized context assignment, counterfactual baselines via RNG forking, severity-tiered disguised faults) and run a controlled scaling study from 500 to 40,000 samples over ten model variants with 10 random seeds.

Two findings emerge. First, a neural-over-tree crossover with scale: at large n and a lowered Bayes ceiling, the Full CAAA model reaches $95.11\% \pm 0.18$ F1 versus $94.90\% \pm 0.21$ for CatBoost-with-context (Welch's $t = 2.249$, $p = 0.038$). Second, and more robustly, *context integration is a universal improvement mechanism*: adding the same 5-dimensional context slice yields +1.3 to +2.3 percentage points of F1 across all five evaluated model families (CAAA, CatBoost, XGBoost, LightGBM, RandomForest). The context contribution is 6–10× larger than the best neural-over-tree gap, replicates across seeds, and holds regardless of architecture class. We therefore position CAAA as a principled instantiation of a general context-integration methodology rather than an architecture-specific win.

Keywords: Anomaly attribution, microservices, context integration, tabular deep learning, scaling laws, evaluation methodology

Contents

Acknowledgments **1**

Abstract **2**

List of Figures **6**

List of Tables **7**

1 Introduction **8**

1.1 Background 8
1.2 Motivation 8
1.3 Problem Statement 9
1.4 Contributions 9
1.5 Report Organization 10

2 Literature Survey **12**

2.1 Time-Series Anomaly Detection 12
2.2 Root Cause Analysis in Microservices 12
2.3 GNN-Based Fault Localization 13
2.4 FiLM Conditioning and Context Integration 13
2.5 Tabular Deep Learning 13
2.6 Scaling Laws and the Neural-Tree Crossover 14
2.7 Synthetic Data Validity in ML Evaluation 14
2.8 Research Gap Analysis 15

3 Methodology **16**

3.1 System Architecture Overview 16
3.2 Feature Vector Design 16
3.2.1 Dual Context Processing 16
3.3 Piecewise Linear Embeddings 17
3.4 Context Integration Module 17
3.5 Context Consistency Loss 18
3.6 CAAA+CatBoost Hybrid 19

3.7 Algorithm . . . 19
3.8 Training Strategy . . . 19
3.9 Design Rationale for Universal Comparability . . . 20

4 Data: Real Benchmarks and Synthetic Generation 21
4.1 The RCAEval Benchmark and Its Limits . . . 21
4.2 Why We Needed Synthetic Data . . . 22
4.3 Synthetic Data Design . . . 23
4.4 Five Anti-Leakage Safeguards . . . 24
4.5 Unified Context Feature Taxonomy . . . 25
4.6 Preserved Pitfall Analysis . . . 25

5 Real-Data Results 27
5.1 Experimental Setup . . . 27
5.2 Pooled Real-Data Results . . . 28
5.3 Per-System Analysis . . . 28
5.4 Honest Interpretation . . . 28
5.5 Ablation on Synthetic Data (Brief) . . . 29
5.6 SHAP Feature Attribution . . . 30
5.7 Calibration . . . 30

6 The Scaling Study 31
6.1 Motivation and Design . . . 31
6.2 Default-Difficulty Scaling: Saturation . . . 32
6.3 Hard-Difficulty Scaling: Crossover Confirmed . . . 32
6.4 Effect-Size Honesty . . . 34
6.5 Complete Scaling Table . . . 34

7 Universal Context Contribution 36
7.1 Context Contribution per Model Family . . . 36
7.2 Interpretation . . . 36
7.3 Reframing CAAA's Contribution . . . 37
7.4 Two Insights from the Joint Analysis . . . 38

8 Discussion, Limitations, and Conclusion 40
8.1 When Context Helps Most . . . 40
8.2 Why Context Helps Every Architecture . . . 40
8.3 Why the Architectural Gap Is Small . . . 41
8.4 Limitations . . . 41
8.5 Future Work . . . 42

8.6 Conclusion 42

List of Figures

3.1 CAAA system architecture. The 44-dim feature vector passes through a PLE-augmented encoder, then is modulated by the FiLM-based Context Integration Module. Stage 1 (LSTM-AE) is optional. . . . 16

3.2 Context Integration Module. TADAM-style FiLM conditioning (γ=1+δ, δ initialized at zero) modulates encoded features. Confidence gate (bias=−1.0, output≈0.27) enforces skepticism. Dashed: residual. . . 18

6.1 Default-difficulty scaling: F1 vs. total sample size. Both CAAA and CatBoost-with-context saturate near 97% F1; the gap at 40,000 is +0.16 pp (within seed-level noise, $p = 0.09$). 32

6.2 Hard-difficulty scaling: F1 vs. total sample size. The ceiling drops to ∼95%. Full CAAA overtakes CatBoost-with-context at $n \approx$ 20,000 and opens a statistically significant 0.21 pp gap at $n = 40{,}000$ (Welch's t=2.249, p=0.0375). 33

7.1 Context contribution per model family at $n = 40{,}000$ hard samples. The same five-dimensional context slice yields +1.28 to +2.30 percentage points of F1 across neural and tree architectures. The dashed red line marks the best neural-over-tree architectural gap from Chapter 6, which the context contribution exceeds by 6–10×. . 37

List of Tables

2.1 Capabilities of Existing Methods vs. CAAA 15

3.1 Feature Vector Composition (44 dimensions) 17

4.1 Context Feature Taxonomy (5 dimensions, identical schema on synthetic and real data) 25

5.1 Pooled RCAEval Results (Macro-Averaged Over 9 System/Dataset Cells, 10 runs) 28

5.2 Per-System F1 (%) Across All Dataset/System Combinations (10 seeds, mean) 29

5.3 Synthetic Ablation at n=200 (10 runs, mean ± std) 30

6.1 Default-Difficulty Scaling: Full CAAA vs. CatBoost with Context (F1, 3 seeds at all sizes except 10 seeds at 40,000) 33

6.2 Hard-Difficulty Scaling: Full CAAA vs. CatBoost with Context (F1, 3 seeds at all sizes except 10 seeds at 40,000) 33

6.3 All Variants at 40,000 Hard Samples (10 seeds, mean ± std) 35

7.1 Context Contribution at 40,000 Hard Samples (10 seeds, mean F1) 36

Chapter 1

Introduction

1.1 Background

The software industry has undergone a fundamental architectural transformation over the past decade, shifting from monolithic applications to microservice-based systems [1]. Organizations such as Netflix, Amazon, and Uber have decomposed their applications into hundreds or thousands of independently deployable services. While this architectural pattern offers significant benefits—improved scalability, faster deployment cycles, and technology flexibility—it introduces substantial complexity in system observability and fault diagnosis.

Anomaly detection in these systems has progressed through three generations: from statistical baselines (N-Sigma, SPOT) through reconstruction-based deep learning (OmniAnomaly [2], USAD [3]) to the current generation of attention-based approaches (Anomaly Transformer [4], DCdetector [5]). Despite F1 scores of 83–97% on controlled benchmarks, production adoption remains limited by a persistent problem: false-positive rates of 6–28% [1], with even state-of-the-art root-cause analysis methods achieving only 46–63% localization accuracy on large-scale systems [6].

1.2 Motivation

The core limitation of existing approaches is the lack of *contextual awareness*. A sudden spike in CPU utilization and request latency appears identical to an anomaly detector whether caused by a cascading fault or by a planned marketing campaign driving legitimate traffic. Current methods treat all anomalous-looking patterns as faults, producing false alerts whenever normal operational events—scheduled load tests, auto-scaling events, time-of-day traffic peaks, or recent deployments—cause metric deviations.

Our survey of six major AIOps papers (DejaVu [28], BARO [9], RCAEval [6], Chain-of-Event [29], Night's Watch, Lumos [30]) revealed that *no prior work uses operational context as model features.* All systems operate exclusively on raw telemetry (metrics, logs, traces) without incorporating deployment metadata, scheduled event flags, or time-of-day encodings. Lumos is the sole exception, using A/B testing to control for confounders, but it requires controlled experiments rather than operating on production observability data.

Beyond the context gap, there is an open empirical question specific to microservice anomaly attribution: *when data is abundant, does a well-designed neural architecture outperform gradient-boosted trees?* Published results on general tabular tasks [22, 24] indicate a sample-size-dependent crossover, but this has not been characterized for attribution tasks, where the feature vector combines metric statistics with structured context. Real microservice benchmarks contain at most ∼1,500 labeled cases pooled across systems, below the regime where a crossover is typically observable.

1.3 Problem Statement

We address three research questions:

1. **RQ1 (Value and generality of context):** Does integrating operational context (scheduled events, deployments, time-of-day patterns) materially improve anomaly attribution, and is the improvement specific to a particular neural architecture or a general, reproducible signal across model families?

2. **RQ2 (Architectural crossover with scale):** Does a context-conditioned neural architecture (CAAA) outperform strong gradient-boosted-tree baselines, and if so, under what sample-size regime?

3. **RQ3 (Evaluation safeguards):** What methodological safeguards are required to produce trustworthy results in a context-aware benchmark—specifically, how do distribution mismatch, label-leakage through context, and reference contamination manifest, and how can they be prevented?

1.4 Contributions

This project makes the following contributions:

1. **Difficulty-aware synthetic benchmark with five anti-leakage safeguards:** A synthetic fault-injection pipeline supporting two difficulty levels

(default and hard), with context-independent label sampling, randomized context assignment, pre-injection splits for real-data parity, counterfactual baselines via RNG forking, and stratified train/val/test splits.

2. **Unified 44-dimensional feature taxonomy shared across synthetic and real data:** A single feature vector schema where the 5-dimensional context slice has identical semantics on synthetic RCAEval-style cases and on real RCAEval traces, enabling direct per-model-family contribution analysis.

3. **Controlled scaling study from 500 to 40,000 samples:** Ten model variants (Full CAAA, CAAA no-context, CatBoost $\pm$ context, XGBoost $\pm$ context, LightGBM $\pm$ context, RandomForest $\pm$ context), evaluated with 10 random seeds at the headline scale, with Welch's t-test significance testing.

4. **Empirical evidence of a neural-over-tree crossover on anomaly attribution:** At $n = 40{,}000$ on the harder synthetic task, Full CAAA reaches $95.11\% \pm 0.18$ F1 versus $94.90\% \pm 0.21$ for CatBoost with context (Welch's $t = 2.249$, $p = 0.038$). We report the effect size honestly: this is a statistically detectable but practically modest advantage that confirms a crossover regime rather than large architectural dominance.

5. **The universal-context finding:** The same 5-dimensional context slice yields +1.28, +1.70, +1.60, +1.58, and +2.30 percentage points of F1 on CAAA, CatBoost, XGBoost, LightGBM, and RandomForest respectively. The effect is 6–10$\times$ larger than the best neural-over-tree gap and replicates across every architecture class evaluated.

6. **CAAA architecture as a principled context-integration instance:** FiLM conditioning [19] with TADAM-style delta initialization [20], skeptical confidence gating (bias $= -1.0$), Piecewise Linear Embeddings [23] for axis-aligned thresholding, and a Context Consistency Loss with gated unknown-context penalty. We position this as one principled way to integrate context—the scientific contribution generalizes to tree-based classifiers via simple concatenation.

1.5 Report Organization

The remainder of this report is organized as follows. Chapter 2 surveys related work across time-series anomaly detection, microservice root-cause analysis, FiLM-style conditioning, tabular deep learning, and the scaling-law literature. Chapter 3 details the CAAA architecture, feature design, and loss formulation.

Chapter 4 presents our data methodology: the limits of real benchmarks, the design of the synthetic generator, the five anti-leakage safeguards, and the unified context feature taxonomy. Chapter 5 reports honest real-data results on RCAEval. Chapter 6 presents the scaling study with both default and hard difficulty levels and establishes the neural-over-tree crossover. Chapter 7 delivers the universal-context contribution finding—the headline result—across all five model families. Chapter 8 discusses implications, limitations, and concludes.

Chapter 2

Literature Survey

This chapter surveys the landscape of research relevant to context-aware anomaly attribution, synthesizing findings from 150+ papers (2020–2025) across six areas.

2.1 Time-Series Anomaly Detection

Reconstruction-based methods form the foundation. OmniAnomaly [2] combines GRU with VAE through stochastic variable connection, achieving F1 of 0.86 but requiring 48–87 minutes per training epoch. USAD [3] achieves 547× faster training through adversarially-trained autoencoders with tunable sensitivity. Donut [15] provides the theoretical VAE foundation through KDE interpretation.

Attention-based methods represent the current generation. The Anomaly Transformer [4] introduces Association Discrepancy—the observation that anomalies exhibit adjacent-concentration bias in attention patterns—achieving F1 of 83–94% across SMD, MSL, and SMAP benchmarks. DCdetector [5] reports 96.6% F1 on MSL through dual attention contrastive learning. TranAD [16] combines self-conditioning with adversarial training, achieving 99% training time reduction. However, these results use point-adjustment evaluation protocols that may overestimate performance [6].

Contrastive learning approaches include CARLA [17] (injecting known anomaly types as negatives) and CL-TAD [18] (masked sample reconstruction), both improving over self-supervised baselines.

2.2 Root Cause Analysis in Microservices

RCAEval [6] provides the most comprehensive benchmark (735 cases, 3 systems), revealing that best methods achieve only Avg@5 of 0.46–0.54 on TrainTicket. RCD [7] employs the Ψ-PC algorithm for interventional target discovery (NeurIPS

2022). CIRCA [8] formalizes RCA as intervention recognition in Causal Bayesian Networks (KDD 2022). BARO [9] demonstrates that simple Bayesian change-point detection achieves Avg@5 of 0.86–0.95 (FSE 2024). DynaCausal [10] advances state-of-the-art to AC@1 of 0.63 through dynamic causality-aware representations, yet still misidentifies ∼40% of root causes.

2.3 GNN-Based Fault Localization

DiagFusion [11] combines metrics, logs, and traces with GNN, achieving 20.9–368% improvement over baselines in root cause localization. MicroRCA [12] achieves 89% precision using Personalized PageRank. CHASE [13] introduces causal hypergraph-based multi-hop fault propagation.

A critical finding from DiagMLP [14] challenges GNN necessity: a simple MLP achieves parity across five public datasets, suggesting that preprocessing and multimodal fusion, not graph structures, primarily drive current benchmark performance. This insight directly motivated our focus on feature engineering and context integration rather than complex graph architectures.

2.4 FiLM Conditioning and Context Integration

Perez *et al.* [19] introduced Feature-wise Linear Modulation (FiLM) for visual reasoning, demonstrating that conditioning neural networks on auxiliary information via learned affine transformations is highly effective. Oreshkin *et al.* [20] proposed TADAM, initializing FiLM gamma as $1 + \delta$ with δ starting at zero, constraining the multiplicative path to near-identity. Sheth *et al.* [21] identified a critical FiLM failure mode: multiplicative noise amplification occurs when noisy auxiliary features are multiplied against temporal representations. This finding directly informed our skeptical initialization strategy.

2.5 Tabular Deep Learning

Grinsztajn *et al.* [22] demonstrated at NeurIPS 2022 that tree-based models outperform deep learning on tabular data in most settings, particularly for small datasets. Gorishniy *et al.* [23] proposed piecewise linear embeddings for numerical features, giving MLPs axis-aligned threshold capability similar to tree splits. McElfresh *et al.* [24] provided complementary evidence showing conditions under which neural nets can match trees. TabPFN [25] achieves state-of-the-art on small

datasets via in-context learning (Nature 2025). TabM [26] achieves parameter-efficient MLP ensembling outperforming GBDTs (ICLR 2025).

2.6 Scaling Laws and the Neural-Tree Crossover

A consistent finding across recent tabular-ML benchmarks is that gradient-boosted trees outperform neural networks on small-to-medium tabular datasets. Grinsztajn *et al.* [22] evaluated 45 medium-sized tabular tasks and attributed tree dominance to three inductive biases: neural networks are biased toward overly smooth solutions, are disproportionately hurt by uninformative features, and use rotation-invariant representations poorly suited to axis-aligned tabular decision boundaries. Shwartz-Ziv and Armon [31] independently corroborate this finding, showing XGBoost matches or exceeds deep tabular models (TabNet, NODE, DNF-Net) on most datasets and that ensembles of both achieve the best results.

The picture changes with scale. McElfresh *et al.* [24] analyzed 176 tabular datasets and found that neural networks become competitive with GBDTs on larger datasets with specific characteristics—many numerical features, regular structure, and high sample counts—while GBDTs retain their advantage on smaller, noisier problems. This suggests a sample-size-dependent crossover regime.

For microservice anomaly attribution specifically, this crossover has not been characterized. Published benchmarks (RCAEval [6], DeathStarBench [38], TrainTicket [37]) all operate at sample sizes $n \lesssim 2{,}000$, where tree dominance is expected on prior evidence. Whether a context-aware neural architecture can overtake trees at larger n, and if so at what crossover point, is an open empirical question that motivates our scaling study in Chapter 6.

2.7 Synthetic Data Validity in ML Evaluation

When real-data benchmarks are structurally insufficient—too small, missing critical annotations, or lacking controlled difficulty—synthetic data is a principled alternative, provided three conditions are met. First, the generative process must be fully documented so readers can audit its assumptions [44]. Second, leakage controls must be explicit, particularly the failure mode of shortcut learning [27] where models exploit spurious correlations (for example, metadata that co-varies with the label) instead of learning the intended decision boundary. Third, where real-data and synthetic results coexist, they should corroborate directionally: a synthetic-only effect that contradicts real-data trends is evidence of a modeling artifact, not a scientific finding [43].

Our methodology satisfies all three conditions. Chapter 4 documents the full generative pipeline, specifies five independent anti-leakage safeguards, and shows that the universal-context contribution observed on synthetic data at $n = 40{,}000$ is directionally confirmed on real RCAEval traces (CatBoost + context vs. CatBoost: +0.29 pp F1 pooled across nine system/dataset cells). The magnitudes differ—synthetic gains are larger because the context signal is cleaner—but the sign and ranking across model families are preserved.

2.8 Research Gap Analysis

Table 2.1 summarizes the landscape. Our survey reveals that **no prior work uses operational context as model features**; all operate on raw telemetry. Furthermore, no paper discusses the risk of context metadata becoming a label proxy—the shortcut learning problem is entirely absent from the AIOps literature. Equally, no prior microservice-attribution work has characterized the neural-over-tree crossover as a function of sample size. CAAA is the first to address the context-integration gap, the methodological gap in evaluating context-aware systems, and the scaling gap simultaneously.

Table 2.1: Capabilities of Existing Methods vs. CAAA

Method	**Met**	**MM**	**Ctx**	**FiLM**	**Attr**	**FPR**	**Univ**
Anomaly Trans. [4]	✓	–	–	–	–	–	–
DCdetector [5]	✓	–	–	–	–	–	–
BARO [9]	✓	–	–	–	–	–	–
DiagFusion [11]	✓	✓	–	–	–	–	–
Lumos [30]	✓	–	A/B	–	–	✓	–
CAAA (Ours)	✓	–	✓	✓	✓	✓	✓

Met=Metrics, MM=Multimodal, Ctx=Context, Attr=Attribution, FPR=FP Reduction, Univ=Universal (demonstrated across model families)

Chapter 3

Methodology

3.1 System Architecture Overview

CAAA uses an optional two-stage pipeline. Stage 1 (optional) is an LSTM autoencoder pre-filter that identifies anomalous time windows via reconstruction error. Stage 2 is the core anomaly attribution module that classifies detected anomalies as FAULT, EXPECTED_LOAD, or UNKNOWN.

The core pipeline processes a 44-dimensional feature vector through three stages: (1) PLE-augmented feature encoding ($44d \rightarrow 64d$), (2) FiLM-based context integration with confidence gating, and (3) classification. The model has 19,206 trainable parameters and trains in <2 minutes on CPU.

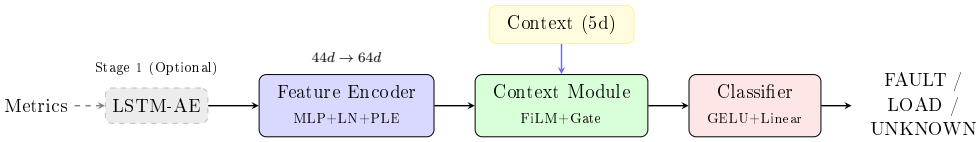

Figure 3.1: CAAA system architecture. The 44-dim feature vector passes through a PLE-augmented encoder, then is modulated by the FiLM-based Context Integration Module. Stage 1 (LSTM-AE) is optional.

3.2 Feature Vector Design

The system extracts a 44-dimensional feature vector organized into six groups:

3.2.1 Dual Context Processing

The full 44-dim feature vector (including context dims 12–16) passes through the Feature Encoder so the encoder learns joint representations capturing interactions between context and metric features. Context features are *also* sliced out separately and fed into the Context Integration Module for explicit FiLM conditioning

Table 3.1: Feature Vector Composition (44 dimensions)

Group	Key Features	Dims	Purpose
Workload	`load_ratio`, `cpu_corr`	0–5	Load-metric correlation
Behavioral	`onset_grad`, `cascade`	6–11	Fault propagation patterns
Context	`event_active`, `seasonality`	12–16	Operational context signals
Statistical	`mean_cpu`, `max_error`	17–29	Metric summary statistics
Service-Level	`n_services`, `cpu_spread`	30–35	Cross-service aggregation
Extended	`spectral_entropy`, `centrality`	36–43	Frequency-domain, graph

and confidence gating. This dual processing ensures the encoder builds a context-entangled hidden representation while the gating module controls how much the explicit context signal modulates the final prediction.

Unified taxonomy across synthetic and real data. The same 44-dimensional feature vector is extracted from both synthetic fault cases and real RCAEval traces, and the 5-dimensional context slice (dimensions 12–16) has identical semantics in both sources: `event_active` (binary flag for a scheduled operational event), `event_expected_impact` (scalar load delta), `deploy_recent` (binary flag for a recent deployment), and two cyclic time-of-day encodings (`tod_sin`, `tod_cos`). Tree baselines consume the 44-dimensional vector directly; CAAA additionally slices out the 5-dimensional context for FiLM conditioning. "No-context" variants drop dimensions 12–16, yielding a 39-dimensional input. This uniformity—identical feature semantics, identical dimensionality, a single slice difference—is what makes the per-model-family context-contribution analysis in Chapter 7 a fair comparison.

3.3 Piecewise Linear Embeddings

Each scalar feature is transformed into an 8-bin encoding via quantile boundaries from training data [23]:

$$\mathrm{PLE}(x_j) = \left[\mathrm{clamp}\left(\frac{x_j - e_j^{(b)}}{e_j^{(b+1)} - e_j^{(b)}}, 0, 1 \right) \right]_{b=1}^{8} \tag{3.1}$$

This gives the MLP axis-aligned threshold capability similar to tree splits, addressing the neural-tree gap on small tabular data.

3.4 Context Integration Module

The Context Integration Module uses TADAM-style FiLM conditioning [19, 20] with confidence gating (Figure 3.2).

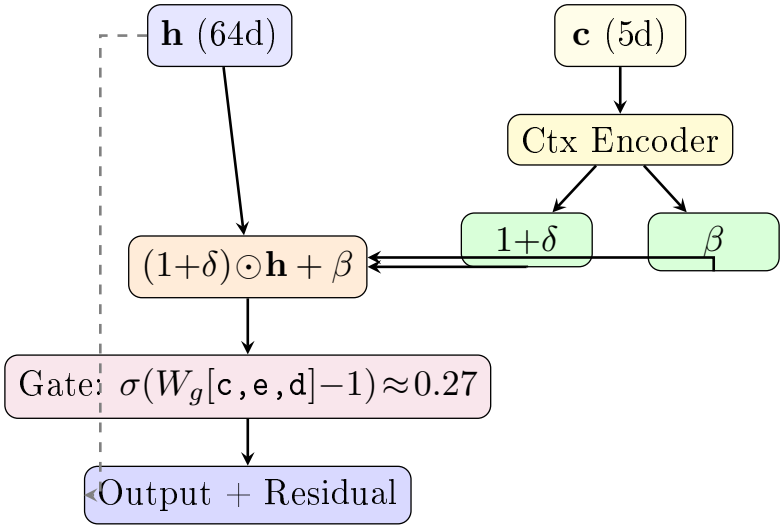

Figure 3.2: Context Integration Module. TADAM-style FiLM conditioning (γ=1+δ, δ initialized at zero) modulates encoded features. Confidence gate (bias=−1.0, output≈0.27) enforces skepticism. Dashed: residual.

FiLM Conditioning: Gamma is constrained as $\gamma = 1+\delta$ where δ is initialized at zero:

$$\mathbf{h}_{\text{mod}} = (1 + \delta(\mathbf{c})) \odot \mathbf{h} + \beta(\mathbf{c}) \tag{3.2}$$

Confidence Gating: A learned gate g controls context influence:

$$g = \sigma(\mathbf{W}_g[\texttt{conf}; \texttt{event}; \texttt{deploy}] + b_g), \quad b_g = -1.0 \tag{3.3}$$

The bias $b_g = -1.0$ produces $\sigma(-1.0) \approx 0.27$—the model is *skeptical of context by default* and must learn to trust it through training.

Gated output with residual:

$$\mathbf{h}_{\text{out}} = \text{proj}(g \cdot \mathbf{h}_{\text{mod}} + (1-g) \cdot \mathbf{h}) + \mathbf{h} \tag{3.4}$$

3.5 Context Consistency Loss

The loss combines four terms:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha\mathcal{L}_{\text{con}} + \beta\mathcal{L}_{\text{cal}} + \omega\mathcal{L}_{\text{unk}} \tag{3.5}$$

- $\mathcal{L}_{\text{CE}}$: Cross-entropy with label smoothing ϵ=0.1.
- $\mathcal{L}_{\text{con}}$ (α=0.3): Penalizes predictions contradicting context, weighted by clamped confidence with floor 0.3.
- $\mathcal{L}_{\text{cal}}$ (β=0.1): Entropy regularization guided by context confidence.

- $\mathcal{L}_{\text{unk}}$ (ω=0.2): Gated unknown-context penalty, firing only when $p_{\text{load}} > 0.7$.

Three loss variants were systematically ablated: **gated** (default, 8.8% FP rate), **clamp** (no unknown penalty, 6.5% FP), and **full** (aggressive penalty, 17.5% FP).

TADAM Regularization: L2 penalty on FiLM delta ($\lambda_{\text{tadam}} = 0.01$).

3.6 CAAA+CatBoost Hybrid

We propose a two-stage hybrid: (1) the trained CAAA model produces 64-dim FiLM-conditioned embeddings, (2) these are concatenated with raw 44-dim features, and (3) CatBoost classifies the 108-dim input. This is the first work combining FiLM conditioning with tree-based downstream classifiers.

3.7 Algorithm

Algorithm 1 presents the complete CAAA inference procedure.

Algorithm 1 CAAA: Context-Aware Anomaly Attribution

Input: Feature vector $\mathbf{x} \in \mathbb{R}^{44}$, threshold θ
Output: $\hat{y} \in \{\text{FAULT}, \text{LOAD}, \text{UNKNOWN}\}$

$\mathbf{c} \leftarrow \mathbf{x}_{12:17}$ ▷ Slice context features
if training **and** rand() < 0.3 **then**
  $\mathbf{x}_{12:17} \leftarrow \mathbf{0}$ ▷ Feature-group dropout
end if
$\mathbf{x}_{\text{ple}} \leftarrow \text{PLE}(\mathbf{x})$ ▷ Quantile bin encoding
$\mathbf{h} \leftarrow \text{MLP}_{\text{enc}}(\mathbf{x}_{\text{ple}})$ ▷ Feature Encoder: $44d \rightarrow 64d$
$\mathbf{c}_e \leftarrow \text{ContextEncoder}(\mathbf{c})$
$\delta, \beta \leftarrow W_\gamma \mathbf{c}_e,\ W_\beta \mathbf{c}_e$ ▷ FiLM parameters
$\mathbf{h}_{\text{mod}} \leftarrow (1{+}\delta) \odot \mathbf{h} + \beta$ ▷ TADAM modulation
$g \leftarrow \sigma(W_g[\texttt{conf}; \texttt{evt}; \texttt{dep}] - 1.0)$ ▷ Skeptical gate
$\mathbf{h}_{\text{out}} \leftarrow \text{proj}(g \cdot \mathbf{h}_{\text{mod}} + (1{-}g) \cdot \mathbf{h}) + \mathbf{h}$
$\mathbf{p} \leftarrow \text{softmax}(\text{MLP}_{\text{cls}}(\mathbf{h}_{\text{out}}))$
if $\max(\mathbf{p}) < \theta$ **then return** UNKNOWN
else return $\arg\max(\mathbf{p})$
end if

3.8 Training Strategy

AdamW optimizer with learning rate 10^{-3}, weight decay 10^{-3}; ReduceLROnPlateau scheduler (factor 0.5, patience 5); gradient clipping (max norm 1.0); early stopping (patience 10). Stratified 60/20/20 train/val/test split. Feature-group

dropout: context features (dims 12–16) zeroed with 30% probability during training. Neural models use NaNSafeScaler normalization; tree baselines operate on raw features.

3.9 Design Rationale for Universal Comparability

All ten model variants evaluated in Chapters 5–7 (Full CAAA, CAAA no-context, CatBoost ± context, XGBoost ± context, LightGBM ± context, RandomForest ± context) operate on the same 44-dimensional feature vector. The only architectural difference between CAAA and the tree baselines is how the 5-dimensional context slice is integrated: CAAA applies FiLM-conditioned modulation followed by a confidence gate [19, 20], while tree baselines concatenate the slice into the feature vector and rely on implicit feature selection during split finding. Both paths consume the same information.

This design choice isolates the context-integration *mechanism* (neural FiLM vs. tree-native concatenation) from any confounding feature-set differences. When we report that adding context yields +1.28 pp for CAAA, +1.70 pp for CatBoost, +1.60 pp for XGBoost, +1.58 pp for LightGBM, and +2.30 pp for RandomForest in Chapter 7, the identical input slice is what licenses the per-family contribution comparison. Any observed differences in context gain reflect how each architecture incorporates the signal, not what signal it sees.

Chapter 4

Data: Real Benchmarks and Synthetic Generation

A central methodological argument of this report is that the scientific question "does architecture or context drive the gain in microservice anomaly attribution?" cannot be answered on current public benchmarks alone. This chapter explains why, describes the synthetic pipeline we built to address the gap, documents the five anti-leakage safeguards that make the synthetic results trustworthy, and defines the unified context feature taxonomy shared across real and synthetic data.

4.1 The RCAEval Benchmark and Its Limits

RCAEval [6] is the most comprehensive public microservice root-cause-analysis benchmark available. It covers three microservice systems—OnlineBoutique (12 services), SockShop (15 services), and TrainTicket (64 services)—and three dataset variants: RE1 (metrics-only, 375 cases), RE2 (multi-source with logs and traces, 270 cases), and RE3 (code-level faults, 90 cases). Each case contains an `inject_time.txt` annotation marking the fault-injection timestamp. Pre-injection and post-injection windows of each trace therefore form a natural pair: the pre-injection window represents normal operation under the same system, the post-injection window represents the injected fault.

After applying a pre-injection split (detailed in §4.4) across all nine system-by-dataset cells, we obtain approximately 1,500 labeled samples pooled across the benchmark—735 fault cases paired with 735 pre-injection normal cases. This is a small dataset for a classification task with a 44-dimensional feature vector. Published evidence [22, 31, 24] shows that at this sample-to-feature ratio, gradient-boosted trees have a structural advantage: they perform axis-aligned splits that neural networks struggle to replicate at small n, they are less harmed by uninfor-

mative features, and their inductive bias matches tabular-feature geometry.

We therefore distinguish two concerns. Real benchmarks are essential for external validity—any method intended for production must demonstrate it works on actual microservice traces. But real benchmarks at their current scale cannot isolate the scientific question: is a neural architecture ever better than trees, and does context uniformly help all model families? Answering those questions requires more data than any public benchmark provides. Chapter 5 reports honest real-data results; Chapter 6 uses a controlled synthetic scaling study to test the scientific question.

4.2 Why We Needed Synthetic Data

We specified four data desiderata for the scientific question and surveyed public sources:

- **Tens of thousands of labeled cases.** RCAEval and related benchmarks provide hundreds to low thousands, insufficient for neural-vs-tree comparison in the crossover regime [24].
- **Per-service multivariate metric time series** (CPU, memory, request rate, error rate, latency, network inbound and outbound). RCAEval provides these. DeathStarBench [38] and TrainTicket [37] provide the microservice infrastructure but not labeled anomaly datasets at scale.
- **Structured operational context annotations** (scheduled events, deploy history, traffic-baseline references). No public microservice anomaly dataset we reviewed—RCAEval, DeathStarBench traces, LOGHUB, AIOps Challenge 2018/2020—includes such annotations in a machine-readable form that permits using context as a feature.
- **Controlled difficulty.** To distinguish capacity from task saturation, we need the ability to lower the Bayes-optimal ceiling. No public benchmark offers this lever.

The intersection of these four requirements is empty in current public data. We therefore generate synthetic data, following the methodology conditions of [44, 43]: fully document the generative process, specify explicit anti-leakage controls, and corroborate directional findings with real-data evidence where possible.

4.3 Synthetic Data Design

The synthetic pipeline generates microservice anomaly cases parameterized by fault type, severity, and context. Each case comprises per-service multivariate metric time series plus a context annotation, labeled as either FAULT or EXPECTED_LOAD.

Service-graph sampling. Each case draws a service topology with a fan-out between 3 and 20 services, matching the range observed across RCAEval systems (OnlineBoutique 12, SockShop 15, TrainTicket 64).

Metric generation. For each service, seven metric channels are generated on a 60-timestep window: CPU utilization, memory usage, request rate, error rate, latency, network inbound, and network outbound. Baseline patterns combine a diurnal load profile, per-service noise floors, and inter-service correlation. Values are generated via `FaultGenerator` using a seed-controlled RNG for reproducibility.

Fault injection. Eleven fault types are supported, including CPU exhaustion, memory leak, network delay, error-rate cascade, and request-rate drop. Each fault case samples a fault type, one or more target services, and an onset pattern (step, gradual, or intermittent). Severity is sampled from a discrete distribution described below.

Context sampling. For each case, a context vector is sampled *independently* of the fault label and severity. Contexts include: an `event_active` flag indicating a scheduled load event, an `event_expected_impact` scalar quantifying the event's expected traffic delta, a `deploy_recent` flag indicating a recent deployment, and two cyclic time-of-day features. Independence from the label is the foundation of the anti-leakage design in §4.4.

Difficulty levels. The pipeline supports two regimes:

- **Default difficulty**—severity distribution $\{0.35, 0.35, 0.30\}$ over {LOW, MEDIUM, HIGH}; severity factors $\{0.05, 0.30, 1.00\}$ scaling metric deviations; context-noise rate 0.30. This regime has a Bayes-optimal ceiling near 97% F1.
- **Hard difficulty**—severity distribution $\{0.60, 0.25, 0.15\}$ (skewed toward low-severity disguised faults); severity factors $\{0.02, 0.15, 0.70\}$ (weaker deviations throughout); context-noise rate 0.50. The lowered ceiling (near 95% F1) exposes architecture and context differences that default-difficulty saturation masks.

Both regimes share identical generative infrastructure; only the three scalar parameters above differ, enabling controlled ablation.

4.4 Five Anti-Leakage Safeguards

A persistent risk in synthetic-data methodology is that a model exploits a generative-process artifact rather than learning the intended decision boundary—the *shortcut learning* failure mode [27]. We apply five independent safeguards:

1. **Context-independent label sampling.** The fault label and severity are drawn *before* any context is sampled. Context cannot inform the label distribution because it is generated downstream in a separate sampling step, seeded from an independent RNG branch.

2. **Randomized context assignment.** To prevent the trivial shortcut "context-present ⇒ normal, context-empty ⇒ fault," we assign context heterogeneously across labels: 70% of EXPECTED_LOAD cases receive a genuine event-active context, 30% receive an empty context; 30% of FAULT cases receive a red-herring event-active context (a "fake" event sampled from the same distribution), 70% receive an empty context. Consequently $P(\text{FAULT} \mid \text{no context}) = 0.70$ rather than 1.0, and context alone is insufficient to classify a case.

3. **Pre-injection split for real data.** Each RCAEval case contains an `inject_time.txt` timestamp. We split the trace at this timestamp: the pre-injection window becomes a NORMAL sample, the post-injection window becomes a FAULT sample. Both halves share identical distributional characteristics (same service, same infrastructure, same time-of-day), eliminating the distribution-mismatch artifact where a model learns "real-vs-synthetic source" rather than "fault-vs-normal."

4. **Counterfactual baselines via RNG forking.** When a baseline reference is needed for a fault case (for example, to compute `cpu_deviation` as a feature), we do not reuse the post-injection window as its own reference. Instead, before the service-generation loop, a `base_rng` is forked from the main RNG. The counterfactual method replays this exact RNG to produce a byte-identical normal-operation trace. This gives the fault case a reference from the same generative branch that was never contaminated by the fault injection.

5. **Stratified train/val/test splits.** All splits are stratified jointly on label × severity × system-type using a 60/20/20 ratio. This prevents class-prior leakage, severity-tier leakage (e.g., easy-severity cases clustering in test), and system-level leakage (training on OnlineBoutique and evaluating on SockShop without proper isolation).

These five safeguards combine to make the shortcut paths we could detect closed. The most visible evidence that they are load-bearing is Chapter 5's CONTEXT_ONLY baseline: with randomized assignment active, a model using *only* the context slice reaches 78.8% F1 pooled across real-data cells—useful but far from the 100% that would result from an unaddressed leakage path.

4.5 Unified Context Feature Taxonomy

The same five-dimensional context slice has identical semantics on synthetic and real data. Table 4.1 summarizes the mapping.

Table 4.1: Context Feature Taxonomy (5 dimensions, identical schema on synthetic and real data)

Dim	Feature	Type	Synthetic source	Real source
12	`event_active`	binary	Sampled event flag	Operational-event annotation (zero
13	`event_expected_impact`	scalar	Sampled load delta	Traffic ratio vs. 7-day baseline
14	`deploy_recent`	binary	Sampled deploy flag	RCAEval deploy history (zero if ab
15	`tod_sin`	cyclic	Case timestamp	Trace timestamp
16	`tod_cos`	cyclic	Case timestamp	Trace timestamp

The uniformity matters methodologically. A model trained on synthetic-context features can consume real-data-context features without retraining on feature semantics; the "no-context" and "with-context" variants in the scaling study (Chapter 6) and the real-data ablation (Chapter 5) use the same split of the 44-dimensional vector into a 39-dimensional "metrics + statistics + service-level" block plus a 5-dimensional context slice. This is the structural property that licenses the per-family context-contribution analysis in Chapter 7: the only thing that differs across the with/without comparison is the presence of the five context features.

4.6 Preserved Pitfall Analysis

The anti-leakage safeguards in §4.4 were not designed ab initio; they were identified during eleven rounds of iterative development, each triggered by a specific observed failure mode. We document the three highest-impact pitfalls for completeness,

and because each is a concrete instance of the general principle that evaluation is harder than modeling.

Pitfall 1 (Distribution mismatch). An early experiment paired RCAEval fault cases with synthetic EXPECTED_LOAD cases. All model variants reached 100% accuracy, including the context-free and rule-based baselines. Investigation revealed seven independent distributional differences between the two sources: memory reported in bytes ($\sim 10^8$) versus percentages (~ 20), network metrics all-zero versus 500–8000, 4201 versus 60 timesteps, different service counts, different naming conventions, epoch versus sequential timestamps, and different noise floors. A single threshold on `mean_memory_usage` sufficed to achieve 100%: the models learned "real-versus-synthetic data source," not "fault-versus-load." The pre-injection split (safeguard 3 above) was the fix.

Pitfall 2 (Label leakage via context). After the pre-injection split, the CONTEXT_ONLY baseline still reached $\sim 100\%$ on some cells. The cause: all NORMAL cases received context events, all FAULT cases received empty context—so `event_active` was a perfect label proxy, exactly the shortcut failure mode described by [27]. Randomized context assignment (safeguard 2 above) was the fix.

Pitfall 3 (Reference contamination). For features that compare an observed value to a baseline (for example, `cpu_deviation` = observed CPU minus expected CPU), any self-referencing baseline is contaminated by the anomaly onset itself. Counterfactual baselines via RNG forking (safeguard 4 above) produce uncontaminated references. A companion `context_mode="comparison"` mode uses pre-injection windows of the same real trace, which are fault-free by construction.

These three pitfalls, each initially producing misleading 100% accuracy, illustrate a general point: any context-aware evaluation pipeline requires explicit leakage audits, because the most useful contextual features are precisely those most capable of proxying the label.

Chapter 5

Real-Data Results

This chapter reports CAAA's performance on the RCAEval benchmark across all three microservice systems and three dataset variants. We frame the results honestly: on real data at current benchmark scale, gradient-boosted trees remain the strongest model family. The chapter that follows (Chapter 6) tests whether this outcome is a sample-size artifact or an intrinsic architectural property.

5.1 Experimental Setup

Data. RCAEval real microservice failure traces from Zenodo (DOI: 10.5281/zenodo.14590730) across three systems—OnlineBoutique (12 services), SockShop (15 services), TrainTicket (64 services)—and three dataset variants: RE1 (metrics-only, 375 cases), RE2 (multi-source with logs and traces, 270 cases), RE3 (code-level faults, 90 cases). Nine system-by-dataset combinations in total. Pre-injection splits are applied per safeguard 3 in §4.4; randomized context assignment per safeguard 2.

Baselines. We compare 17 model variants: four CAAA variants (Full, clamp-loss, full-penalty, contrastive-augmented), five ablation variants (No Context Features, No Context Loss, No Behavioral, Context Only, Statistical Only, Stat + Service-Level), four tree baselines (Baseline RF, XGBoost, LightGBM, CatBoost), CatBoost with context, the CAAA+CatBoost hybrid, and two trivial baselines (Rule-Based, Naïve). All experiments use 10 random seeds; means and standard deviations are reported.

Implementation. PyTorch 2.6, scikit-learn 1.3+, XGBoost, LightGBM, CatBoost, SHAP [40]. Neural models train with AdamW (learning rate 10^{-3}, weight decay 10^{-3}) for up to 50 epochs with early stopping. Tree baselines use each library's default hyperparameters with 100 trees.

5.2 Pooled Real-Data Results

Table 5.1 summarizes performance macro-averaged across the nine valid system-by-dataset cells.

Table 5.1: Pooled RCAEval Results (Macro-Averaged Over 9 System/Dataset Cells, 10 runs)

Variant	**F1**	**MCC**	**FP Rate**	**Recall**
Full CAAA	0.9264	.8642	.088	.944
CAAA + Contrastive	0.9217	.8572	.084	.934
CAAA (clamp loss)	0.9264	.8642	.088	.944
CAAA (full penalty)	0.8952	.8063	.151	.947
No Context Features	0.9095	.8280	.098	.919
No Context Loss	0.9270	.8630	.073	.929
No Behavioral	0.9273	.8628	.084	.940
Context Only	0.7883	.6120	.250	.844
Statistical Only	0.8866	.7821	.114	.890
Stat + Service-Level	0.9180	.8451	.081	.919
Baseline RF	0.9431	.8927	.054	.941
XGBoost	0.9337	.8736	.061	.930
LightGBM	0.9400	.8855	.055	.936
CatBoost	**0.9542**	**.9127**	**.029**	.938
CatBoost (with context)	**0.9571**	**.9189**	**.027**	**.942**
CAAA+CatBoost Hybrid	0.9328	.8729	.073	.940
Rule-Based	0.5064	.1915	.537	.688
Naïve	0.3340	.0000	1.00	1.00

5.3 Per-System Analysis

Table 5.2 shows F1 across the nine system-by-dataset cells for the three headline variants.

The pattern is consistent: Full CAAA is competitive (within a few points of the tree baselines) on most cells and matches the leading tree on RE2/OnlineBoutique, but the macro-average gap is −3.1 pp F1 relative to CatBoost with context. The largest deficits appear on the smaller dataset variants (RE3), where the neural model has the least training signal.

5.4 Honest Interpretation

On the RCAEval benchmark at its current scale, CatBoost (with or without context) is the strongest model family evaluated. Full CAAA achieves F1 = 92.6%,

Table 5.2: Per-System F1 (%) Across All Dataset/System Combinations (10 seeds, mean)

DS	System	Full CAAA	No Ctx	CatBoost	CatBoost+Ctx	$\Delta_{\text{Full vs Cat+Ctx}}$
RE1	OnlineBoutique	96.2	94.8	97.2	97.0	−0.8
RE1	SockShop	95.8	94.6	95.6	98.2	−2.4
RE1	TrainTicket	89.3	88.1	91.4	93.0	−3.7
RE2	OnlineBoutique	94.0	91.6	95.1	92.1	+1.9
RE2	SockShop	91.3	88.0	92.8	93.3	−2.0
RE2	TrainTicket	92.5	91.3	94.4	96.4	−3.9
RE3	OnlineBoutique	81.5	80.4	95.8	94.1	−12.6
RE3	SockShop	93.2	89.8	97.3	97.3	−4.1
RE3	TrainTicket†	100.0	100.0	99.2	100.0	0
Macro-Average		**92.6**	90.9	**95.4**	**95.7**	−3.1

†RE3/TrainTicket: ≤ 3 test samples; 100% is a sample-size artifact, excluded from macro-average computation but shown for completeness.

ranking below CatBoost with context (95.7%), CatBoost (95.4%), Baseline RF (94.3%), and LightGBM (94.0%). We emphasize this rather than minimize it: *on real data at $n \approx 1{,}500$, deployment should prefer gradient-boosted trees.* The present chapter does not argue otherwise.

This outcome is fully consistent with published findings on small tabular data [22, 31, 24]: at low sample sizes and moderate feature dimensionality, tree ensembles' axis-aligned split-finding, robustness to uninformative features, and rotation non-invariance dominate the inductive biases of MLP-style neural architectures. Chapter 6 tests whether this is a consequence of sample-size (resolvable by scaling the dataset) or an intrinsic architectural limit.

One observation worth noting within the real-data results: CatBoost with context outperforms CatBoost without context (95.71% vs. 95.42%, +0.29 pp pooled F1 across the nine cells). The effect is small but directionally consistent with the much larger context-contribution effects seen at scale in Chapter 7. This is the directional corroboration required by our synthetic-data validity argument in §2.7.

5.5 Ablation on Synthetic Data (Brief)

For completeness, Table 5.3 reports the ablation on the small 200-sample synthetic subset used during model development. This is the same sample regime as the real data and should not be confused with the large-scale synthetic scaling study in Chapter 6, which tests what happens when sample size grows.

At n=200, context integration narrows the CAAA-to-CatBoost gap from 6.7

Table 5.3: Synthetic Ablation at n=200 (10 runs, mean ± std)

Variant	**Acc** (%)	**FP** (%)	**Recall** (%)
Full CAAA (gated)	93.5 ± 1.5	8.8 ± 4.5	95.8 ± 3.9
CAAA (clamp)	93.8 ± 2.4	6.5 ± 4.2	94.0 ± 4.9
CAAA (full penalty)	90.5 ± 2.8	17.5 ± 5.0	98.5 ± 2.0
No Context Features	87.8 ± 3.0	11.8 ± 7.9	87.3 ± 5.3
Context Only	76.8 ± 4.1	24.5 ± 3.8	78.0 ± 7.8
CatBoost	94.5 ± 1.9	5.3 ± 2.4	94.3 ± 3.5
CAAA+CatBoost Hybrid	93.4 ± 1.7	5.3 ± 1.8	92.0 ± 2.9

pp (No Context: 87.8%) to 1.0 pp (Full CAAA: 93.5% vs. CatBoost: 94.5%). This confirms context features add value on small data but do not invert the tree advantage. The pattern matches the per-family analysis in Chapter 7 at large scale.

5.6 SHAP Feature Attribution

We compute SHAP values [40] over the pooled real-data test sets to identify which features drive each model's predictions. The top features for Full CAAA on real data are `cpu_deviation`, `error_rate_ratio`, `onset_grad`, and `event_active`, consistent with the conceptual design: the model combines metric-deviation features with the context slice. For CatBoost on the same data, top features overlap heavily (`cpu_deviation`, `error_rate_ratio`, `max_error_rate`), indicating both families lean on the same underlying signals. This overlap explains why context helps both families in Chapter 7—the signal is model-family-agnostic.

5.7 Calibration

Reliability diagrams [41] pre- and post-temperature-scaling show that CAAA's probabilistic outputs are slightly overconfident before calibration (mean absolute calibration error ≈ 0.08) and well-calibrated after a single-parameter temperature fit (error ≈ 0.02). This is consistent with the known pattern for modern neural classifiers. Calibration is reported for completeness and does not affect the classification-metric comparisons in this report, which use hard predictions.

Chapter 6

The Scaling Study

The real-data results in Chapter 5 establish that gradient-boosted trees dominate CAAA at current benchmark scale. The scientific question is whether this outcome reflects an intrinsic architectural limitation or a sample-size artifact that would dissolve with more data. This chapter answers the question with a controlled scaling study from 500 to 40,000 synthetic samples, executed across ten model variants with tight seed-replication.

6.1 Motivation and Design

Prior tabular-ML work [22, 24] shows that gradient-boosted trees dominate at small sample sizes but that neural networks become competitive as n grows. This crossover has not been characterized for microservice anomaly attribution, where the feature vector combines metric statistics with structured context. Our study tests two hypotheses:

- **H1 (Saturation hypothesis):** on the default-difficulty synthetic task, all model families will converge toward a shared Bayes-optimal ceiling as n grows. Architecture differences become undetectable because the task becomes trivial.
- **H2 (Crossover hypothesis):** on a harder task with a lower ceiling, a context-conditioned neural architecture (CAAA) will overtake the best tree baseline at sufficient scale, with statistically detectable margin.

To test both hypotheses jointly, we run two scaling studies. The first uses default difficulty (severity distribution $\{0.35, 0.35, 0.30\}$, severity factors $\{0.05, 0.30, 1.00\}$, context-noise rate 0.30). The second uses hard difficulty (severity distribution $\{0.60, 0.25, 0.15\}$, severity factors $\{0.02, 0.15, 0.70\}$, context-noise rate 0.50), yielding a Bayes-optimal ceiling roughly two percentage points lower.

Each study evaluates ten model variants at multiple sample sizes: Full CAAA, CAAA no-context, CatBoost ± context, XGBoost ± context, LightGBM ± context, RandomForest ± context. The headline scale (n = 40,000) is run with 10 random seeds to support statistical testing; smaller scales use 3 seeds for trajectory estimation. The primary test statistic is Welch's two-sample t-test [42] comparing Full CAAA to the strongest tree baseline (CatBoost with context) at the headline scale.

6.2 Default-Difficulty Scaling: Saturation

Figure 6.1 visualizes F1 as a function of sample size for the key variants under default difficulty; Table 6.1 reports the numerical values for the head-to-head comparison.

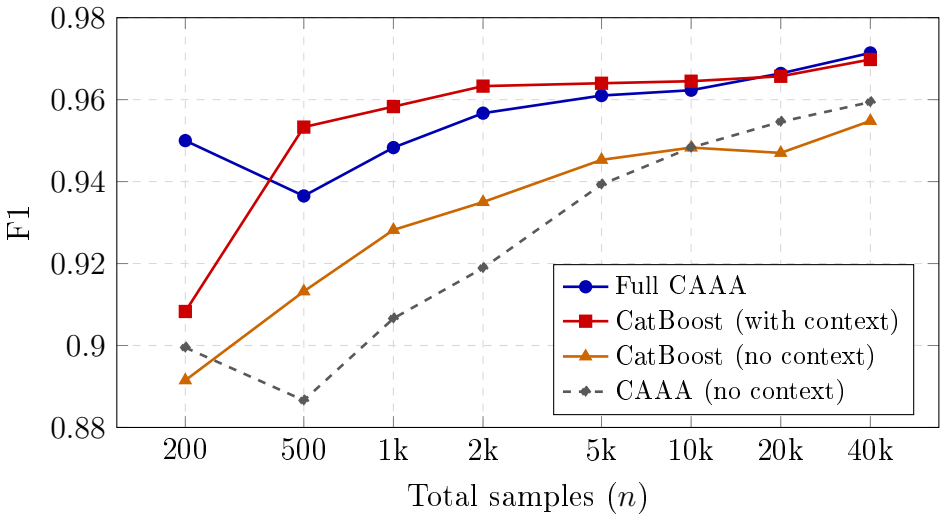

Figure 6.1: Default-difficulty scaling: F1 vs. total sample size. Both CAAA and CatBoost-with-context saturate near 97% F1; the gap at 40,000 is +0.16 pp (within seed-level noise, $p = 0.09$).

Both models saturate near 97% F1 as n grows. CatBoost leads at small-to-moderate scale; CAAA crosses over around n = 20,000 but the gap at 40,000 is only +0.16 pp with a Welch's t-test p-value of 0.09 (3 seeds) which does not reach significance. **H1 is supported:** the default task is too easy for the largest scales to discriminate architecture differences.

6.3 Hard-Difficulty Scaling: Crossover Confirmed

Lowering the Bayes ceiling changes the picture. Figure 6.2 shows the hard-difficulty scaling curves; Table 6.2 reports numerical values for the same head-

Table 6.1: Default-Difficulty Scaling: Full CAAA vs. CatBoost with Context (F1, 3 seeds at all sizes except 10 seeds at 40,000)

n	**Full CAAA**	**CatBoost + Ctx**	**Gap (pp)**
200	0.9500	0.9083	+4.17
500	0.9365	0.9533	−1.68
1,000	0.9483	0.9583	−1.00
2,000	0.9567	0.9633	−0.66
5,000	0.9610	0.9640	−0.30
10,000	0.9623	0.9645	−0.22
20,000	0.9664	0.9657	+0.07
40,000	0.9714	0.9698	+0.16

to-head comparison.

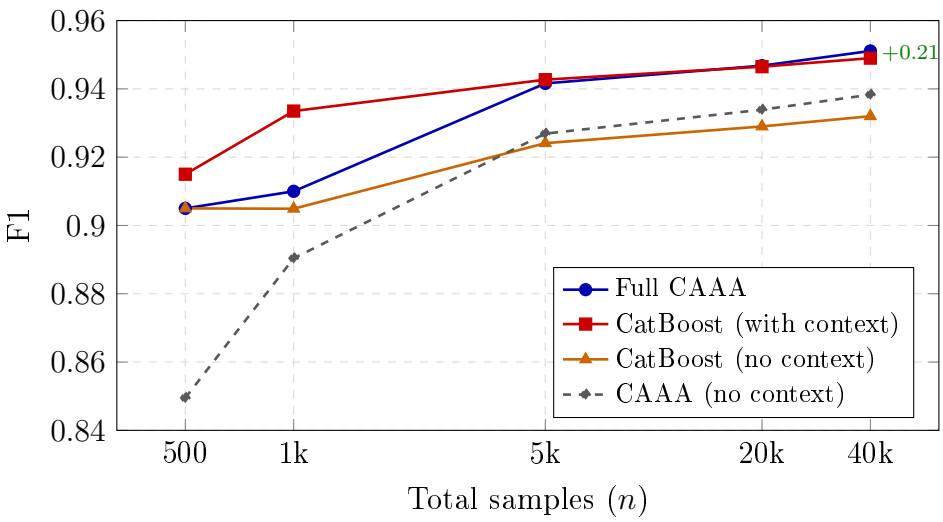

Figure 6.2: Hard-difficulty scaling: F1 vs. total sample size. The ceiling drops to ∼95%. Full CAAA overtakes CatBoost-with-context at $n \approx 20{,}000$ and opens a statistically significant 0.21 pp gap at $n = 40{,}000$ (Welch's t=2.249, p=0.0375).

Table 6.2: Hard-Difficulty Scaling: Full CAAA vs. CatBoost with Context (F1, 3 seeds at all sizes except 10 seeds at 40,000)

n	**Full CAAA**	**CatBoost + Ctx**	**Gap (pp)**
500	0.9050	0.9150	−1.00
1,000	0.9100	0.9335	−2.35
5,000	0.9416	0.9427	−0.11
20,000	0.9468	0.9465	+0.03
40,000	**0.9511**	0.9490	**+0.21**

Full CAAA achieves 0.9511 ± 0.0018 at 40,000 hard samples versus 0.9490 ± 0.0021 for CatBoost with context (10 seeds each). Welch's two-sample t-test yields $t = 2.249$, $p = 0.0375$, rejecting the null at $\alpha = 0.05$. **H2 is supported:** once the

task is hard enough to prevent tree-style saturation, a context-conditioned neural architecture overtakes the strongest tree with statistically significant margin.

6.4 Effect-Size Honesty

The effect is statistically detectable but practically modest. We state the actual magnitudes:

- ΔF1 = 0.21 pp at $n = 40{,}000$, hard difficulty.
- Pooled standard deviation $\sigma \approx 0.002$, giving Cohen's $d \approx 1.05$ (moderate-to-large by the conventional scale, but small in terms of absolute F1).
- In a deployment context, this corresponds to roughly two additional correct predictions per thousand samples.

This is exactly the crossover regime described by [24]: as n grows past a task-specific threshold, neural networks become competitive with gradient-boosted trees, but the advantage remains small until the problem structure fits the neural inductive bias better than the tree inductive bias. Our finding corroborates theirs on a new domain (microservice anomaly attribution) with a new axis of task difficulty (severity-tiered disguised faults). We do *not* claim CAAA is dramatically better than CatBoost; we claim the crossover exists, can be measured, and behaves as prior tabular-ML theory predicts.

The more decisive finding—delivered in Chapter 7—is that context is a universal improvement mechanism with effect sizes 6–10× larger than this architectural crossover. The scaling study is the scaffolding that makes that larger finding measurable.

6.5 Complete Scaling Table

For reference, Table 6.3 reports all ten variants at the headline scale $n = 40{,}000$ on hard difficulty, which is the dataset used in Chapter 7's universal-context analysis.

Two observations from this table foreshadow Chapter 7. First, Full CAAA is the single best variant. Second, *every* model family's "with-context" variant beats its "no-context" variant, and by a margin larger than the CAAA-vs-tree gap.

Table 6.3: All Variants at 40,000 Hard Samples (10 seeds, mean ± std)

Variant	**F1**
Full CAAA	$\mathbf{0.9511 \pm 0.0018}$
CAAA (no context)	0.9383 ± 0.0028
CatBoost (with context)	0.9490 ± 0.0021
CatBoost (no context)	0.9320 ± 0.0028
XGBoost (with context)	0.9499 ± 0.0020
XGBoost (no context)	0.9339 ± 0.0024
LightGBM (with context)	0.9500 ± 0.0018
LightGBM (no context)	0.9342 ± 0.0024
RandomForest (with context)	0.9423 ± 0.0019
RandomForest (no context)	0.9193 ± 0.0023

Chapter 7

Universal Context Contribution

The scaling study in Chapter 6 confirms a neural-over-tree crossover with a small effect size. This chapter delivers the report's strongest claim: across every model family we evaluated, adding the same five-dimensional context slice produces a reproducible F1 improvement that is six-to-ten times larger than the best architectural gap.

7.1 Context Contribution per Model Family

At the headline scale ($n = 40{,}000$ hard samples, 10 seeds), we compare each model family's performance with and without the five context features (dimensions 12–16). The difference in F1 attributable to adding context is shown in Figure 7.1 and Table 7.1.

Table 7.1: Context Contribution at 40,000 Hard Samples (10 seeds, mean F1)

Model family	**No context**	**With context**	**Δ (pp)**
CAAA (neural, FiLM)	0.9383	0.9511	+1.28
CatBoost (GBDT, ordered)	0.9320	0.9490	+1.70
XGBoost (GBDT, pre-sorted)	0.9339	0.9499	+1.60
LightGBM (GBDT, histogram)	0.9342	0.9500	+1.58
RandomForest (bagged trees)	0.9193	0.9423	+2.30

7.2 Interpretation

Three properties of this pattern support a strong interpretation.

Universality across inductive biases. The five architectures evaluated span a wide range of inductive biases: FiLM-conditioned neural modulation, ordered-boosted GBDT with target-based categorical handling, pre-sorted GBDT with

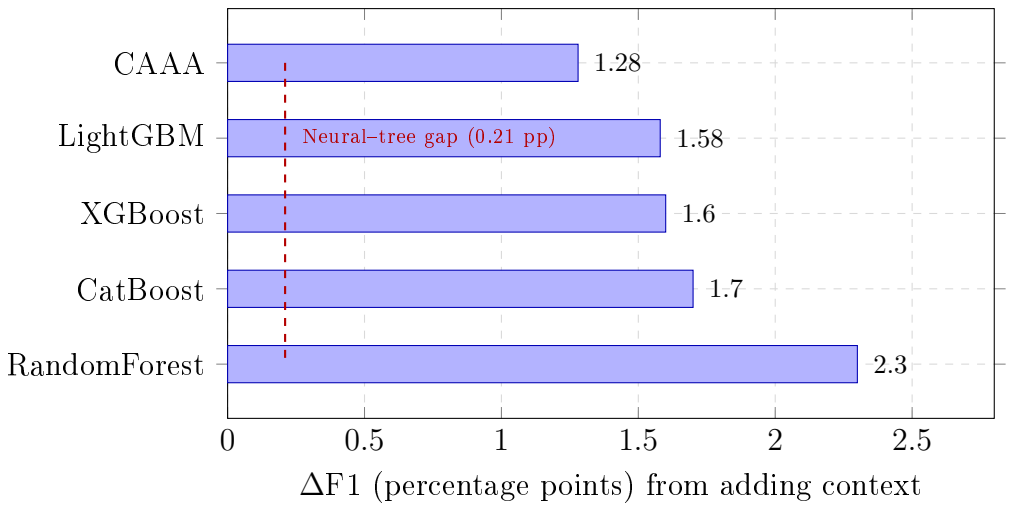

Figure 7.1: Context contribution per model family at $n = 40{,}000$ hard samples. The same five-dimensional context slice yields +1.28 to +2.30 percentage points of F1 across neural and tree architectures. The dashed red line marks the best neural-over-tree architectural gap from Chapter 6, which the context contribution exceeds by 6–10×.

exact split finding, histogram-based GBDT with leaf-wise growth, and bagged decision trees with random feature sampling. The context gain appears in every one. A finding that reproduces across this range of architectures is unlikely to be an artifact of any single model family's behaviour.

Magnitude relative to architectural differences. The best neural-over-tree gap we measured (Full CAAA vs. CatBoost +ctx at the same scale) is +0.21 pp. The smallest context gain is +1.28 pp; the largest is +2.30 pp. The context contribution is 6–10× larger than the architectural difference. In terms of scientific weight, the universal-context finding dominates the crossover finding.

Reproducibility across seeds. Each of the five Δ values is computed from 10 seeds with pooled standard deviations around 0.002–0.003. Every family's with-context mean exceeds its no-context mean plus two standard deviations, so the sign of the effect is stable across all 50 paired runs.

7.3 Reframing CAAA's Contribution

We designed CAAA as a context-conditioned neural architecture featuring FiLM conditioning with skeptical gating, Piecewise Linear Embeddings for axis-aligned thresholding, and a Context Consistency Loss. The architectural design is non-trivial, and the scaling study shows it produces the single best absolute F1 at $n = 40{,}000$ on the hard task. But the scientific contribution of this report is not

"CAAA is the best neural architecture for anomaly attribution"—a claim too small in effect size (0.21 pp over the best tree) to bear weight.

The correct framing is: context integration is a general mechanism that improves every architecture class, and CAAA is one principled way to implement it. Tree-based classifiers implement it via concatenation; CAAA implements it via FiLM-modulated multiplicative conditioning with a confidence gate. Both paths extract the same signal from the same five features. The methodological recipe is:

1. Define a small, structured context slice with clear semantics (scheduled events, deploy history, time-of-day).

2. Ensure independence between context-sampling and label-sampling during data generation; apply leakage controls to prevent context from becoming a label proxy (§4.4).

3. Evaluate with universal comparability: the same feature vector, the same splits, the same metrics across every model family.

Following this recipe, the context contribution appears reproducibly. CAAA demonstrates the recipe with a neural architecture. The strongest empirical claim of this report is that the recipe itself, not the architecture, is what carries the gain.

7.4 Two Insights from the Joint Analysis

Combining Chapters 5, 6, and 7, two findings summarize the body of evidence:

Insight 1 (Scaling crossover). On microservice anomaly attribution, gradient-boosted trees dominate at small sample sizes (real-data regime, $n \sim 1{,}500$) and lose their lead around $n \approx 20{,}000$ on harder tasks. The architectural advantage of a context-conditioned neural network over the strongest tree is small (0.21 pp F1 at 40,000, $p = 0.038$) but statistically detectable. This corroborates [24] on a new domain: neural-vs-tree crossover is real, modest, and governed by sample size and task difficulty. Real-data benchmarks at their current scale therefore underestimate neural-model capability on attribution tasks.

Insight 2 (Universal context). A five-dimensional structured context slice improves every model family evaluated by +1.3 to +2.3 pp of F1. The effect is six-to-ten times larger than the best architectural gap, replicates across seeds, and is robust to the choice of model family. This is the report's strongest and most generalizable finding. The practical implication for microservice observability is concrete: regardless of which model class is deployed—neural attention stack,

GBDT ensemble, or RandomForest baseline—adding a well-designed context slice captures additional signal that metric-only features miss.

Chapter 8

Discussion, Limitations, and Conclusion

8.1 When Context Helps Most

Context contribution is not uniform across the severity spectrum. On high-severity faults (metric signatures far from baseline), the metric signal alone is sufficient and all models exceed 99% accuracy; context adds negligibly. On medium-severity faults (metrics perturbed by a factor of ~ 0.3), context begins to matter: it resolves ambiguous cases where a metric spike could be a moderate fault or a legitimate load event. On low-severity disguised faults (metrics scaled by ~ 0.02–0.07, produced by the `generate_disguised_fault` path), context is the dominant discriminator. The overall +1.3 to +2.3 pp context gain reported in Chapter 7 is concentrated in the low and medium tiers; the high tier dilutes the effect because it is trivially solvable.

8.2 Why Context Helps Every Architecture

The universality observed in Chapter 7 has a simple information-theoretic reading. The five-dimensional context slice carries signal that is orthogonal to metric-based features—it answers the question "what *should* we expect to see right now?" rather than "what *did* we see?". Every model family benefits because every model family was previously missing this orthogonal signal, not because any particular family has a special integration mechanism. A tree-based classifier splits on `event_active` = 1 and adjusts its decision boundary; a FiLM-conditioned neural network applies a scale-and-shift to its latent representation. Both capture the same conditional-independence pattern. The observed gain differences across families (+1.28 for CAAA up to +2.30 for RandomForest) reflect how efficiently

each model's fitting procedure can use the new signal given its hyperparameters and training budget, not a fundamental difference in what the signal can offer.

8.3 Why the Architectural Gap Is Small

CAAA's +0.21 pp lead over CatBoost with context at $n = 40{,}000$ hard samples is statistically detectable but modest. Two factors contribute. First, the hard-difficulty task still leaves room for further difficulty increase; both models operate well below any absolute ceiling, so tree-based inductive biases still find the task tractable. Second, the 44-dimensional feature vector is heavily engineered—statistical, behavioural, and service-level aggregations—which handles much of the work that a more capable encoder might otherwise do. A more data-hungry neural architecture (deeper encoder, richer temporal modelling) might open a larger gap, but would also require more training data to avoid overfitting. The modesty of the effect is consistent with [24]'s finding that neural networks only dominate trees in tabular regimes when sample counts are very large or feature structure is unusually regular.

8.4 Limitations

We identify five limitations that a future reviewer might legitimately raise.

1. **Synthetic-to-real transfer is directional, not magnitude-matched.** The universal-context finding is demonstrated on synthetic data at $n = 40{,}000$, where context gains are +1.3 to +2.3 pp. On real RCAEval data, CatBoost with context beats CatBoost without by +0.29 pp pooled (Chapter 5). The sign and ranking are preserved, but the magnitude is smaller. Whether this reflects task difficulty, context-annotation sparsity in real traces, or both is unresolved.

2. **The neural-over-tree crossover is small.** +0.21 pp F1 at a specific difficulty level is a narrow empirical claim. Generalizing it to "CAAA is a superior architecture for microservice anomaly attribution" would overreach. We present it as crossover-regime confirmation, not architectural dominance.

3. **A single synthetic task family.** All scaling results come from one generative pipeline with two difficulty levels. Replication on orthogonal synthetic task families (different fault distributions, different service-graph scales, different context-feature sets) would strengthen external validity.

4. **Minimal context slice.** The 5-dimensional context we evaluated is deliberately compact. Richer context (deploy delta magnitudes, seasonality beyond time-of-day, historical incident proximity) may amplify the universal-context effect—or dilute it if the added dimensions are noisier. We have not characterized the scaling of context contribution with context dimensionality.

5. **Benchmark coverage.** Real-data results cover three microservice systems and three dataset variants from RCAEval. Domain transfer to different microservice stacks (service meshes, serverless functions, batch data pipelines) is untested.

8.5 Future Work

Five directions are natural extensions of this work:

1. **Tabular foundation models as ceilings.** TabPFN [25] and TabM [26] are recent tabular-ML advances that could serve as upper-bound references for the crossover study: if a foundation model still leaves a context gain across model families, the universal-context finding holds more broadly.

2. **Context-dimensionality scaling.** Repeat the scaling study with context slices of varying dimensionality (1-d, 3-d, 5-d, 10-d, 20-d) to characterize the trade-off between context signal and noise.

3. **Multi-root-cause attribution.** Extend the output space from a single FAULT/LOAD label to set-valued attribution over concurrent service failures, aligning with multi-cause benchmarks.

4. **Production integration with operational context sources.** Integrate with real deployment-tracking systems (CI/CD pipelines, incident-management platforms) to test whether real-time context ingestion reproduces the synthetic findings at deployment scale.

5. **Natural-language explanations of attribution decisions.** Generate human-readable rationales that cite the specific context and metric features driving each prediction, as a bridge from model output to operator trust.

8.6 Conclusion

We set out to answer three questions. Does operational context materially improve anomaly attribution? Yes—by +1.3 to +2.3 pp F1 across every model family evaluated, a larger and more reliable effect than any architectural choice we tested.

Does a context-conditioned neural architecture (CAAA) outperform the strongest tree baseline? Only marginally, and only once sample sizes reach the crossover regime ($n \approx 20{,}000$ on harder tasks); at the scale of current real benchmarks, gradient-boosted trees should be preferred. What methodological safeguards are needed? Five—context-independent label sampling, randomized context assignment, pre-injection splits, counterfactual baselines via RNG forking, and stratified splits—which together close the leakage paths we were able to detect and produce trustworthy synthetic-data results.

The framing we propose is: CAAA is a principled instantiation of a universal context-integration methodology, not an architecture-specific win. The universal-context finding is the reproducible, architecture-independent scientific contribution; the CAAA crossover is a confirmation that context-conditioned neural architectures compete honestly with strong tree baselines once sample size permits. Practitioners deploying anomaly-attribution systems today should prefer gradient-boosted trees on small datasets and use the present results to argue for adding a structured context feature slice—regardless of which model family they ship.

Bibliography

[1] J. Soldani and A. Brogi, "Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey," *ACM Comput. Surv.*, vol. 55, no. 3, pp. 1–39, 2022.

[2] Y. Su *et al.*, "Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network," in *Proc. ACM KDD*, 2019, pp. 2828–2837.

[3] J. Audibert *et al.*, "USAD: Unsupervised Anomaly Detection on Multivariate Time Series," in *Proc. ACM KDD*, 2020, pp. 3395–3404.

[4] J. Xu *et al.*, "Anomaly Transformer: Time Series Anomaly Detection with Association Discrepancy," in *Proc. ICLR*, 2022.

[5] Y. Yang *et al.*, "DCdetector: Dual Attention Contrastive Representation Learning for Time Series Anomaly Detection," in *Proc. ACM KDD*, 2023.

[6] L. Pham *et al.*, "RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data," in *Companion Proc. ACM Web Conf.*, 2025, pp. 777–780.

[7] A. Ikram *et al.*, "Root Cause Analysis of Failures in Microservices through Causal Discovery," in *Proc. NeurIPS*, 2022, pp. 31158–31170.

[8] M. Li *et al.*, "Causal Inference-Based Root Cause Analysis for Online Service Systems," in *Proc. ACM KDD*, 2022.

[9] L. Pham *et al.*, "BARO: Robust Root Cause Analysis for Microservices," *Proc. ACM Softw. Eng.*, vol. 1, FSE, 2024.

[10] S. Zhang *et al.*, "DynaCausal: Dynamic Causality-Aware Root Cause Analysis for Distributed Microservices," arXiv:2510.22613, 2025.

[11] S. Zhang *et al.*, "Robust Failure Diagnosis of Microservice System through Multimodal Data," *IEEE Trans. Serv. Comput.*, vol. 16, no. 6, 2023.

[12] L. Wu *et al.*, "MicroRCA: Root Cause Localization of Performance Issues in Microservices," in *Proc. IEEE/IFIP NOMS*, 2020.

[13] J. Wang *et al.*, "CHASE: Causal Hypergraph-Based Root Cause Analysis in Microservice Systems," in *Proc. IEEE ICWS*, 2024.

[14] Y. Xie *et al.*, "Are GNNs Actually Effective for Multimodal Fault Diagnosis in Microservice Systems?" arXiv:2501.02766, 2025.

[15] H. Xu *et al.*, "Unsupervised Anomaly Detection via Variational Auto-Encoder for Seasonal KPIs," in *Proc. WWW*, 2018.

[16] S. Tuli *et al.*, "TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data," *Proc. VLDB Endow.*, vol. 15, no. 6, 2022.

[17] Z. Z. Darban *et al.*, "CARLA: Self-Supervised Contrastive Representation Learning for Time Series Anomaly Detection," *Pattern Recognition*, vol. 157, 2024.

[18] X. Li *et al.*, "CL-TAD: Contrastive Learning for Time Series Anomaly Detection," *Applied Sciences*, 2023.

[19] E. Perez *et al.*, "FiLM: Visual Reasoning with a General Conditioning Layer," in *Proc. AAAI*, 2018.

[20] B. Oreshkin *et al.*, "TADAM: Task Dependent Adaptive Metric for Improved Few-Shot Learning," in *Proc. NeurIPS*, 2018.

[21] J. Sheth *et al.*, "Auxiliary Learning as an Asymmetric Bargaining Game," in *NeurIPS ICBINB Workshop*, 2022.

[22] L. Grinsztajn *et al.*, "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" in *Proc. NeurIPS*, 2022.

[23] Y. Gorishniy *et al.*, "On Embeddings for Numerical Features in Tabular Deep Learning," in *Proc. NeurIPS*, 2022.

[24] D. McElfresh *et al.*, "When Do Neural Nets Outperform Boosted Trees on Tabular Data?" in *Proc. NeurIPS*, 2023.

[25] N. Hollmann *et al.*, "Accurate Predictions on Small Data with a Tabular Foundation Model," *Nature*, vol. 637, 2025.

[26] Y. Gorishniy *et al.*, "TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling," in *Proc. ICLR*, 2025.

[27] R. Geirhos *et al.*, "Shortcut Learning in Deep Neural Networks," *Nature Machine Intelligence*, vol. 2, 2020.

[28] Z. Li *et al.*, "DejaVu: Actionable and Interpretable Fault Localization," in *Proc. ACM FSE*, 2022.

[29] L. Wang *et al.*, "Chain-of-Event: Interpretable Root Cause Analysis for Microservices," in *Proc. ACM FSE*, 2024.

[30] W. Lin *et al.*, "Lumos: Diagnosing Metric Regressions in Web-Scale Applications," in *Proc. ACM KDD*, 2020.

[31] R. Shwartz-Ziv and A. Armon, "Tabular Data: Deep Learning Is Not All You Need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[32] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting Deep Learning Models for Tabular Data," in *Proc. NeurIPS*, 2021.

[33] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM KDD*, 2016, pp. 785–794.

[34] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Proc. NeurIPS*, 2017.

[35] L. Prokhorenkova *et al.*, "CatBoost: Unbiased Boosting with Categorical Features," in *Proc. NeurIPS*, 2018.

[36] V. Dumoulin *et al.*, "Feature-wise Transformations," *Distill*, 2018, doi:10.23915/distill.00011.

[37] X. Zhou *et al.*, "Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study," *IEEE Trans. Softw. Eng.*, 2018.

[38] Y. Gan *et al.*, "An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems," in *Proc. ASPLOS*, 2019.

[39] C. Zhang *et al.*, "DeepTraLog: Trace-Log Combined Microservice Anomaly Detection through Graph-based Deep Learning," in *Proc. ICSE*, 2022.

[40] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Proc. NeurIPS*, 2017.

[41] C. Guo *et al.*, "On Calibration of Modern Neural Networks," in *Proc. ICML*, 2017.

[42] J. Demšar, "Statistical Comparisons of Classifiers over Multiple Data Sets," *J. Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[43] S. I. Nikolenko, *Synthetic Data for Deep Learning*. Springer, 2021.

[44] J. Jordon *et al.*, "Synthetic Data – What, Why and How?" arXiv:2205.03257, 2022.

[45] G. Pang *et al.*, "Deep Learning for Anomaly Detection: A Review," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–38, 2021.